

Final Assessment-Alternative Assessment

Semester	202520	Division	CIS
Assessment title in Syllabus	Final_Project_Portfolio	Program	Bachelor of Information Technology/ Bachelor of Information Systems

Course Code	CDS 2413		
Course Title	INTRODUCTION TO DATA SCIENCE		
Course Instructor		CRN	
Assessment	Project Deliverable 2 – Machine Learning Model Report	Submission Date	Week 13

Student Declaration:

Academic Integrity Statement

In accordance with the HCT Academic Integrity Policy

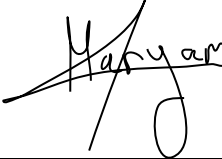
- Students are required to refrain from all forms of academic integrity breaches as defined and explained by HCT.
- A student found guilty of having committed acts of academic integrity breach(es) will be subject to the relevant sanctions as outlined by HCT.

إفادة النزاهة الأكاديمية

وفقاً لسياسة كليات التقنية العليا للنزاهة الأكاديمية

- على الطلبة الالتزام بلوائح وقواعد النزاهة الأكاديمية، كما هو مبين وموضح في السياسات والإجراءات الخاصة بكليات التقنية العليا.
- في حالة ارتكاب الطالب أي شكل من أشكال الإخلال بالنزاهة الأكاديمية، سيتعرض الى العقوبات الموضحة في السياسات ذات الصلة.

This assignment is entirely my/our own work except where I have duly acknowledged other sources in the text and listed those sources at the end of the assignment. I have not previously submitted this work to the HCT, or any other entity. I understand that I may be orally examined on my submission.

Student HCT ID(s):	H00532802	H00533010	H00	H00
Student Name(s):	Medeya Al Hajri	Maryam Al Hashmi		
Student Signature				

Chatbot (e.g. ChatGPT) statement

This submission allows you to use AI generated tools, but the prompt used must be stated and the response must be in red font. Include References to ChatGPT.

Instructions

Remove instructions and hints from the template before submitting the final report to BBLearn.

CLO3: Machine Learning Model (25 Points)

Preprocessing(5 Points)

Data Collection (1 Point)

In this project, the breast cancer dataset is used, which was obtained from **Kaggle**, a popular platform for data science datasets. The dataset is stored in Excel format and contains information about tumor characteristics such as radius, texture, area, and other measurements. To work with the dataset, the **Pandas library** is used to load and manage the data in Python. The dataset includes many features, which are considered the **independent variables (X)**. The variable we want to predict is **diagnosis**, which shows whether the tumor is **malignant (M)** or **benign (B)**. Therefore, diagnosis is the **dependent variable (y)**.

```
import pandas as pd
# Load dataset (fix header issue)
df = pd.read_excel("breast cancer data.xlsx", header=1)
```

```
# Define variables
X = df.drop(columns=['diagnosis'])
y = df['diagnosis']
```

Data Cleaning (1 Point)

Data cleaning is performed to make sure the dataset is accurate and ready for analysis. First, the dataset is checked for **missing values**, because missing data can affect model performance. If any missing values are found, they are replaced using the **mean of the column**.

Next, the dataset is checked for **duplicate rows**, and duplicates are removed to avoid repeated data.

Also, unnecessary columns such as **ID** are removed because they do not help in predicting the diagnosis. This improves the quality of the dataset.

```
# Remove unnecessary columns
if 'id' in df.columns:
    df.drop(columns=['id'], inplace=True)

if 'Unnamed: 32' in df.columns:
    df.drop(columns=['Unnamed: 32'], inplace=True)
# Check missing values
df.isnull().sum()

# Fill missing values with mean
df.fillna(df.mean(numeric_only=True), inplace=True)

# Remove duplicate rows
df.drop_duplicates(inplace=True)
```

Treating the Categorical Variables (1 Point)

The variable **diagnosis** is a categorical variable because it contains text values (M and B). Machine learning models cannot work with text directly, so it needs to be converted into numbers.

In this project, **label encoding** is used:

- Malignant (M) to 1
- Benign (B) to 0

This method is suitable because the variable has only two categories. The rest of the variables are already numerical.

```
from sklearn.preprocessing import LabelEncoder

# Encode diagnosis into numbers
le = LabelEncoder()
df['diagnosis_num'] = le.fit_transform(df['diagnosis'])

# Update X and y
X = df.drop(columns=['diagnosis', 'diagnosis_num'])
y = df['diagnosis_num']
```

Splitting the Dataset (1 Point)

The dataset is divided into training and testing sets to evaluate model performance. The training set is used to build the model, while the testing set is used to check how well the model performs on new data.

In this project, the dataset is split into:

- 80% training data
- 20% testing data

This helps prevent overfitting and gives a more realistic evaluation of the model.

```
from sklearn.model_selection import train_test_split

# Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Featuring the Dataset (1 Point)

Feature scaling is applied to the dataset to ensure that all variables are on a similar scale. This step is important because the dataset contains many features with different ranges, such as

area, radius, and texture. If the data is not scaled, features with larger values may dominate the model and affect its performance.

In this project, **standardization** is used as the feature scaling technique. Standardization transforms the data so that each feature has a **mean of 0** and a **standard deviation of 1**. This helps improve the performance of machine learning models, especially those that depend on distance calculations, such as K-Nearest Neighbors (KNN).

Feature scaling ensures that all features contribute equally to the model and improves the overall accuracy and reliability of the results.

from sklearn.preprocessing import StandardScaler

```
# Apply feature scaling using standardization  
sc = StandardScaler()
```

```
# Fit and transform training data  
X_train = sc.fit_transform(X_train)
```

```
# Transform testing data  
X_test = sc.transform(X_test)
```

Regression Model (7 Points)

Building the Regression Model (6 points)

In this section, regression models are applied to predict the continuous feature (**area_mean**) based on tumor characteristics. Since regression is used for predicting numerical values, the feature **area_mean** is selected as the target variable. This feature represents the average area of tumor cells and is suitable for regression analysis.

The independent variables (X) include all other tumor-related numerical features, while the dependent variable (y) is **area_mean**. The categorical variables (diagnosis and diagnosis_num) are excluded from the model because regression algorithms require numerical inputs only and are not used to predict categories.

Several regression models studied in the lab sessions are applied, including:

- Linear Regression
- Polynomial Regression
- Ridge Regression
- Lasso Regression
- Elastic Net Regression

The models are evaluated using:

- **Mean Squared Error (MSE)** to measure prediction error
- **R-squared (R²)** to measure how well the model explains the data

The best model is selected based on the **highest R^2 value**, as it indicates better prediction performance.

1. Loading and Preprocessing the Data

In this step, the breast cancer dataset is prepared for regression analysis. Since regression is used to predict numerical values, the feature **area_mean** is selected as the dependent variable because it is a continuous numerical feature in the dataset.

The independent variables include the other tumor-related numerical features. The categorical columns **diagnosis** and **diagnosis_num** are removed from the input variables because they are not used as targets in this regression task.

Before applying the models, the dataset is also checked for missing values and unnecessary columns such as **id** and **Unnamed: 32** are removed. This ensures that the data is clean and suitable for regression modeling.

```
import pandas as pd

# Load dataset

df = pd.read_excel("breast cancer data.xlsx", header=1)


# Remove unnecessary columns

if 'id' in df.columns:

    df.drop(columns=['id'], inplace=True)


if 'Unnamed: 32' in df.columns:

    df.drop(columns=['Unnamed: 32'], inplace=True)


# Fill missing values with mean

df.fillna(df.mean(numeric_only=True), inplace=True)


# Define X and y

X = df.drop(columns=['area_mean', 'diagnosis', 'diagnosis_num'])

y = df['area_mean']
```

2. Detecting and Removing Outliers

Outliers are extreme values that are very different from the rest of the data. In regression, outliers can affect the regression line and reduce the accuracy of the model, because the model may try to fit unusual values instead of the general pattern in the dataset.

To reduce this problem, the **Interquartile Range (IQR)** method is used. First, the first quartile (**Q1**) and the third quartile (**Q3**) are calculated for each feature. Then, the IQR is found by subtracting Q1 from Q3. Any values that fall below **$Q1 - 1.5 \times IQR$** or above **$Q3 + 1.5 \times IQR$** are considered outliers.

After detecting the outliers, the rows containing these extreme values are removed from **X**. The target variable **y** is also updated using the same row indexes so that both **X** and **y** still match correctly. This step helps improve the quality of the dataset and makes the regression models more stable.

```
Q1 = X.quantile(0.25)
Q3 = X.quantile(0.75)
IQR = Q3 - Q1
X = X[~((X < (Q1 - 1.5 * IQR)) | (X > (Q3 + 1.5 * IQR))).any(axis=1)]
y = y[X.index]
```

3. Splitting the Data

After preprocessing and removing outliers, the dataset is divided into two parts: a **training set** and a **testing set**. The training set is used to build the regression models, while the testing set is used to evaluate how well the models perform on unseen data.

In this project, the dataset is split into:

- **80% training data**
- **20% testing data**

This is a common ratio because it gives enough data for the model to learn while still keeping a portion for evaluation. Splitting the data is important because it helps prevent overfitting and gives a more realistic measure of model performance.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

4. Applying Regression Models

In this step, different regression models are applied to predict the selected target variable, **area_mean**. Using more than one model is helpful because each regression method works differently, and comparing them allows us to identify which model fits the dataset better.

The models used in this project are:

- Linear Regression
- Polynomial Regression
- Ridge Regression
- Lasso Regression
- Elastic Net Regression

Each model is trained using the training data and then used to predict the values of the testing data. These predicted values are later compared with the actual values using evaluation metrics such as MSE and R^2 .

Linear Regression:

Linear Regression is the simplest regression model. It assumes that the relationship between the independent variables and the target variable is linear. This model is useful as a baseline because it shows how well a straight-line relationship fits the data.

```
from sklearn.linear_model import LinearRegression

# Train the Linear Regression model and predict the test values

lr = LinearRegression()

lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)
```

Polynomial Regression:

Polynomial Regression is used when the relationship between the variables is not completely linear. It creates additional polynomial terms, which allows the model to capture more complex patterns in the data.

In this project, a polynomial degree of 2 is used. This means that squared terms are added to the model, which can improve performance if the relationship between variables is curved rather than straight.

```
from sklearn.preprocessing import PolynomialFeatures

# Transform the data into polynomial features and train the model

poly = PolynomialFeatures(degree=2)

X_train_poly = poly.fit_transform(X_train)

X_test_poly = poly.transform(X_test)


lr_poly = LinearRegression()

lr_poly.fit(X_train_poly, y_train)

y_pred_poly = lr_poly.predict(X_test_poly)
```

Ridge Regression:

Ridge Regression is similar to Linear Regression, but it adds **L2 regularization**. This regularization helps reduce the effect of very large coefficients and improves model stability, especially when the dataset contains many related features.

Ridge Regression is useful when there is a risk of overfitting, because it makes the model less sensitive to small changes in the data.

```
from sklearn.linear_model import Ridge

# Train the Ridge Regression model and predict the test values

ridge = Ridge(alpha=1.0)

ridge.fit(X_train, y_train)

y_pred_ridge = ridge.predict(X_test)
```


Lasso Regression:

Lasso Regression uses **L1 regularization**. Like Ridge Regression, it helps control overfitting, but it also has another advantage: it can reduce some coefficients to zero. This means Lasso can also perform a type of feature selection by keeping the most important variables and reducing the effect of less useful ones.

This can make the model simpler and easier to interpret.

```
from sklearn.linear_model import Lasso

# Train the Lasso Regression model and predict the test values

lasso = Lasso(alpha=0.01)

lasso.fit(X_train, y_train)

y_pred_lasso = lasso.predict(X_test)
```

Elastic Net Regression:

Elastic Net Regression combines both **L1** and **L2** regularization. This means it has some benefits of both Ridge and Lasso Regression. It is useful when the dataset has many features and when we want both regularization and some feature selection.

Because the breast cancer dataset contains many tumor measurement features, Elastic Net is a suitable model to include in the comparison.

```
from sklearn.linear_model import ElasticNet

# Train the Elastic Net Regression model and predict the test values

elastic = ElasticNet(alpha=0.01, l1_ratio=0.5)

elastic.fit(X_train, y_train)

y_pred_elastic = elastic.predict(X_test)
```

5. Evaluating Models

After training all regression models, their performance must be evaluated. In this project, two evaluation metrics are used:

- **Mean Squared Error (MSE):** measures the average squared difference between actual values and predicted values. A lower MSE means better performance.
- **R-squared (R^2):** shows how well the model explains the variation in the target variable. A higher R^2 means a better fit.

```
[12] from sklearn.metrics import mean_squared_error, r2_score
import pandas as pd

# Compare all regression models using MSE and R-squared
results = pd.DataFrame({
    'Model': ['Linear', 'Polynomial', 'Ridge', 'Lasso', 'ElasticNet'],
    'MSE': [
        mean_squared_error(y_test, y_pred_lr),
        mean_squared_error(y_test, y_pred_poly),
        mean_squared_error(y_test, y_pred_ridge),
        mean_squared_error(y_test, y_pred_lasso),
        mean_squared_error(y_test, y_pred_elastic)
    ],
    'R2 Score': [
        r2_score(y_test, y_pred_lr),
        r2_score(y_test, y_pred_poly),
        r2_score(y_test, y_pred_ridge),
        r2_score(y_test, y_pred_lasso),
        r2_score(y_test, y_pred_elastic)
    ]
})

print(results)
```

	Model	MSE	R2 Score
0	Linear	130.068552	0.997706
1	Polynomial	15.955561	0.999719
2	Ridge	106.523448	0.998122
3	Lasso	118.791250	0.997905
4	ElasticNet	107.647274	0.998102

6. Selecting the Best Model

The best regression model is selected based on the **highest R^2 score**. This is because a higher R^2 value means that the model explains a larger portion of the variation in the target variable.

After comparing all models, the model with the highest R^2 score is chosen as the best-performing model for this dataset.

```
[13] ✓ 0s # Select the model with the highest R-squared value
best_model = results.loc[results['R2 Score'].idxmax()]
print(best_model)
```

...	Model	Polynomial
	MSE	15.955561
	R2 Score	0.999719
	Name:	1, dtype: object

Based on the evaluation results, the Polynomial Regression model is selected as the best-performing model. It achieved the lowest Mean Squared Error (15.96) and the highest R-squared score (0.9997), indicating that it provides the most accurate predictions and fits the data extremely well compared to the other models.

This also suggests that the relationship between the features and **area_mean** is non-linear.

7. Printing the Regression Equation

The Linear Regression model produced an intercept value of **-206.90**, which represents the baseline prediction when all input features are zero. The coefficients show how each feature contributes to predicting the target variable (area_mean). Positive coefficients indicate that an increase in the feature value increases the predicted output, while negative coefficients indicate an inverse relationship.

Since the dataset contains many features, the regression equation includes multiple coefficients, each representing the influence of a specific tumor characteristic. This confirms that the prediction of area_mean depends on a combination of several tumor features rather than a single variable.

```
[14] ✓ 0s # Print the intercept and coefficients of the Linear Regression model
a = lr.intercept_
b = lr.coef_

print(a)
print(b)
```

```
-206.90004411609232
[ 1.15917439e+02  6.14527875e-01 -3.29075216e+00 -3.34519164e+02
 -8.41853170e+01 -9.63626500e+01  4.57154714e+02 -4.78549443e+01
  5.08975975e+02  1.93456249e+02  3.57157054e+00  5.01674623e+00
 -2.40617119e+00  2.48637470e+02  3.49698461e+02  2.57796657e+02
 -7.55933374e+02 -6.00628106e+02 -1.09321329e+03 -6.34995324e+01
 -6.69103546e-01 -5.93603844e-01  6.71474135e-01  2.39440166e+02
 -1.77422552e+01  2.58574828e+00 -7.84593579e+01  8.12740142e+01
  4.35630318e+01]
```

The general form of the regression equation used in this project is:

$$\text{area_mean} = a + b_1X_1 + b_2X_2 + \dots + b_nX_n$$

where a represents the intercept and $b_1, b_2, \dots, b_n$ represent the coefficients of the tumor-related features. This equation can be used to manually calculate the predicted values of **area_mean** based on the input features.

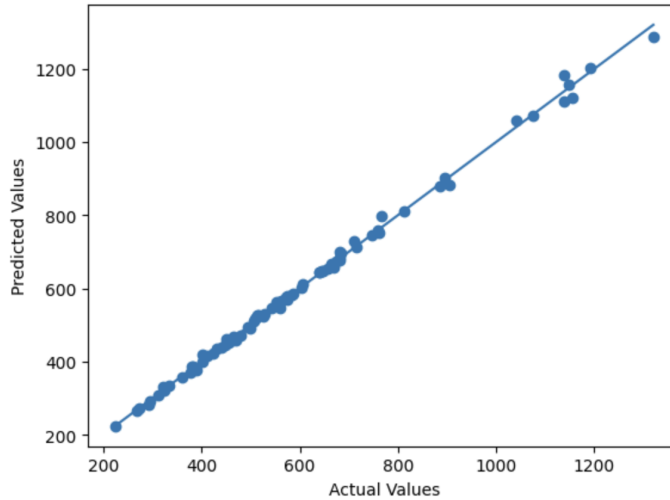
8. Visualization

A scatter plot is used to compare the **actual values** and **predicted values** from the Linear Regression model. This helps visually assess how close the predictions are to the true values.

A diagonal line is also added to represent perfect prediction. Points that are close to this line indicate better model performance, while points farther away show prediction errors.

[17]
✓ Os

```
import matplotlib.pyplot as plt
# Plot the actual values against the predicted values
plt.scatter(y_test, y_pred_lr)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)])
plt.show()
```



The scatter plot shows the relationship between the actual values and the predicted values from the Linear Regression model. Most of the data points are very close to the diagonal line, which represents perfect predictions. This indicates that the model is performing very well and the predicted values are highly accurate. The strong alignment of points along the line confirms that there is a strong relationship between the input features and the target variable (area_mean).

Classification Model (8 Points)

Classifying the dataset based on the target variable (1 Point)

In this section, multiple classification models are applied to predict whether a tumor is malignant or benign based on the given features.

The dataset is classified based on the diagnosis of tumors, where the goal is to determine whether a tumor is malignant or benign. Therefore, this problem is a binary classification problem.

The original dataset contains the diagnosis column with categorical values:

- M = Malignant / B = Benign

During the preprocessing stage, this variable was converted into numeric form by creating a new column called `diagnosis_num`, where:

- Malignant (M) was encoded as **1** And Benign (B) was encoded as **0**

This conversion was done using label encoding in the preprocessing section. The encoded variable `diagnosis_num` is used as the target variable (y) for the classification models, while all other features are used as independent variables (X).

This ensures that the machine learning models can correctly process the data and classify tumors based on their characteristics.

The performance of each classification model is evaluated using a confusion matrix, the number of errors (False Positives and False Negatives), and accuracy.

Bulding classification models (6 Points)

Logistic Regression Model

```
[6]
✓ Os
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# Create the model
log_reg = LogisticRegression()

# Train the model
log_reg.fit(X_train, y_train)

# Make predictions
y_pred_lr = log_reg.predict(X_test)

# Evaluate the model
accuracy_lr = accuracy_score(y_test, y_pred_lr)
print("Logistic Regression Accuracy:", accuracy_lr)

# Confusion matrix
cm_lr = confusion_matrix(y_test, y_pred_lr)
print("Confusion Matrix:\n", cm_lr)

...
Logistic Regression Accuracy: 0.9649122807017544
Confusion Matrix:
[[71  1]
 [ 3 39]]
```

Logistic Regression is used to predict the probability of a binary outcome. In this project, it is used to classify tumors as malignant or benign based on the input features.

K-Nearest Neighbors (KNN) Model

```
[7]
✓ Os
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Create the model (K = 5 as used in slides)
knn = KNeighborsClassifier(n_neighbors=5)

# Train the model
knn.fit(X_train, y_train)

# Make predictions
y_pred_knn = knn.predict(X_test)

# Evaluate the model
accuracy_knn = accuracy_score(y_test, y_pred_knn)
print("KNN Accuracy:", accuracy_knn)

# Confusion matrix
cm_knn = confusion_matrix(y_test, y_pred_knn)
print("Confusion Matrix:\n", cm_knn)

KNN Accuracy: 0.956140350877193
Confusion Matrix:
[[71  1]
 [ 4 38]]
```

K-Nearest Neighbors (KNN) classifies data based on the closest data points. It assigns a class based on the majority of the nearest neighbors.

Naïve Bayes Model

```
[9]
✓ Os
from sklearn.naive_bayes import GaussianNB

# Create model
nb = GaussianNB()

# Train model
nb.fit(X_train, y_train)

# Predict
y_pred_nb = nb.predict(X_test)

# Evaluate
accuracy_nb = accuracy_score(y_test, y_pred_nb)
print("Naive Bayes Accuracy:", accuracy_nb)

# Confusion matrix
cm_nb = confusion_matrix(y_test, y_pred_nb)
print("Confusion Matrix:\n", cm_nb)

... Naive Bayes Accuracy: 0.9210526315789473
Confusion Matrix:
[[69  3]
 [ 6 36]]
```

Naïve Bayes is a probabilistic classifier based on Bayes' theorem. It assumes independence between features and is efficient for classification tasks.

Decision Tree Model

```
[8]
✓ Os
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Create the model
dt = DecisionTreeClassifier(random_state=42)

# Train the model
dt.fit(X_train, y_train)

# Make predictions
y_pred_dt = dt.predict(X_test)

# Evaluate the model
accuracy_dt = accuracy_score(y_test, y_pred_dt)
print("Decision Tree Accuracy:", accuracy_dt)

# Confusion matrix
cm_dt = confusion_matrix(y_test, y_pred_dt)
print("Confusion Matrix:\n", cm_dt)

... Decision Tree Accuracy: 0.9298245614035088
Confusion Matrix:
[[68  4]
 [ 4 38]]
```

Decision Tree is a model that makes decisions based on a tree-like structure. It splits the data into branches based on feature values to classify the target variable.

	Model	Confusion Matrix	Errors (FP + FN)	Accuracy
1	Logistic Regression	[[71 1] [3 39]]	4	0.9649
2	KNN	[[71 1] [4 38]]	5	0.9561
3	Naive Bayes	[[69 3] [6 36]]	9	0.9210
4	Decision Tree	[[68 4] [4 38]]	8	0.9298

Based on the results, the Logistic Regression model achieved the highest accuracy and the lowest number of errors. Therefore, it is selected as the best-performing model for this dataset.

No of elements on the target variable (1 Point)

To understand the distribution of the target variable, the number of elements in each category was calculated. The target variable used in this project is **diagnosis_num**, where:

- **0 represents Benign tumors**
- **1 represents Malignant tumors**

By counting the number of occurrences of each class, we can determine whether the dataset is balanced or imbalanced. This is important because an imbalanced dataset can affect the performance of classification models.


```
[11] ✓ 0s # Count number of elements in each class
      print(y.value_counts())

... diagnosis_num
0      357
1      212
Name: count, dtype: int64
```

The dataset contains:

- Benign (0): 357 samples
- Malignant (1): 212 samples

Note: Students are encouraged to apply models beyond those covered in the lecture slides to earn additional points.

Cluster Analysis (5 Points)

Building the Cluster Analysis(3 Points)

Clustering is an unsupervised learning technique used to group similar data points together without using a target variable. Unlike classification, clustering does not predict a label such as malignant or benign. Instead, it helps identify hidden patterns in the dataset based on similarity between records.

In this project, clustering is applied to the breast cancer dataset using two numerical features: **area_mean** and **radius_mean**. These two features are selected because they are strongly related to tumor size and are suitable for visualizing the clusters on a two-dimensional graph.

Two clustering models are used:

- **K-Means Clustering**
- **Hierarchical Clustering**

For K-Means, the **Elbow Method** is used to identify the optimal number of clusters. For Hierarchical Clustering, a **dendrogram** is used to determine the most suitable number of clusters. Using both methods provides a better understanding of the structure of the dataset and helps earn additional marks beyond the lecture slides.

K-Means Clustering

a. Importing the libraries

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

b. Importing the dataset

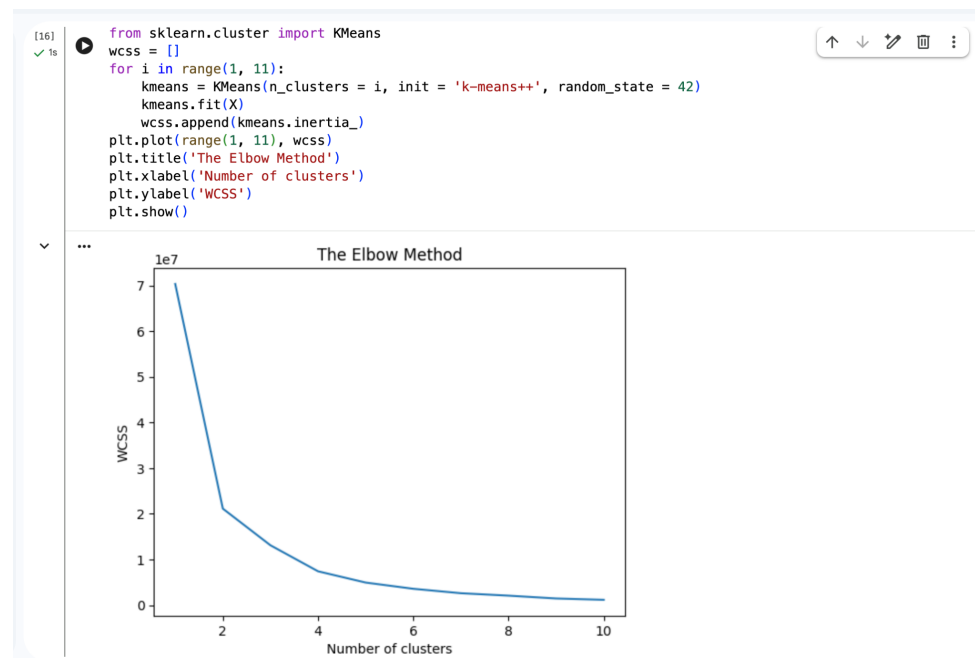
The breast cancer dataset is loaded, and two features are selected for clustering: **area_mean** and **radius_mean**. These are used as the independent variables for grouping the tumor records.

```
[15] # Load the breast cancer dataset and select two features for clustering
dataset = pd.read_excel("breast cancer data.xlsx", header=1)
X = dataset[['area_mean', 'radius_mean']].values
print(X)
```

```
... [[1001.    17.99]
      [1326.    20.57]
      [1203.    19.69]
      ...
      [ 858.1    16.6 ]
      [1265.    20.6 ]
      [ 181.     7.76]]
```

c. Using the elbow method to find the optimal number of clusters

The Elbow Method is used to determine the best number of clusters. It calculates the **Within-Cluster Sum of Squares (WCSS)** for different values of K. The optimal number of clusters is chosen where the graph starts to bend and the reduction in WCSS becomes slower.



The Elbow Method graph shows that the rate of decrease in WCSS becomes slower after K = 3, forming a clear “elbow” point. This indicates that adding more clusters beyond this point does not significantly improve the model. Therefore, the optimal number of clusters for this dataset is selected as 3.

d. Apply the K-Means model on the dataset

After analyzing the elbow graph, the optimal number of clusters is selected. In this project, the selected number of clusters is 3. The K-Means model is then applied to assign each record to a cluster.

```
[17] # Apply the K-Means model using the selected number of clusters
✓ Os kmeans = KMeans(n_clusters = 3, init = 'k-means++', random_state = 42)
y_kmeans = kmeans.fit_predict(X)
print(y_kmeans)
```

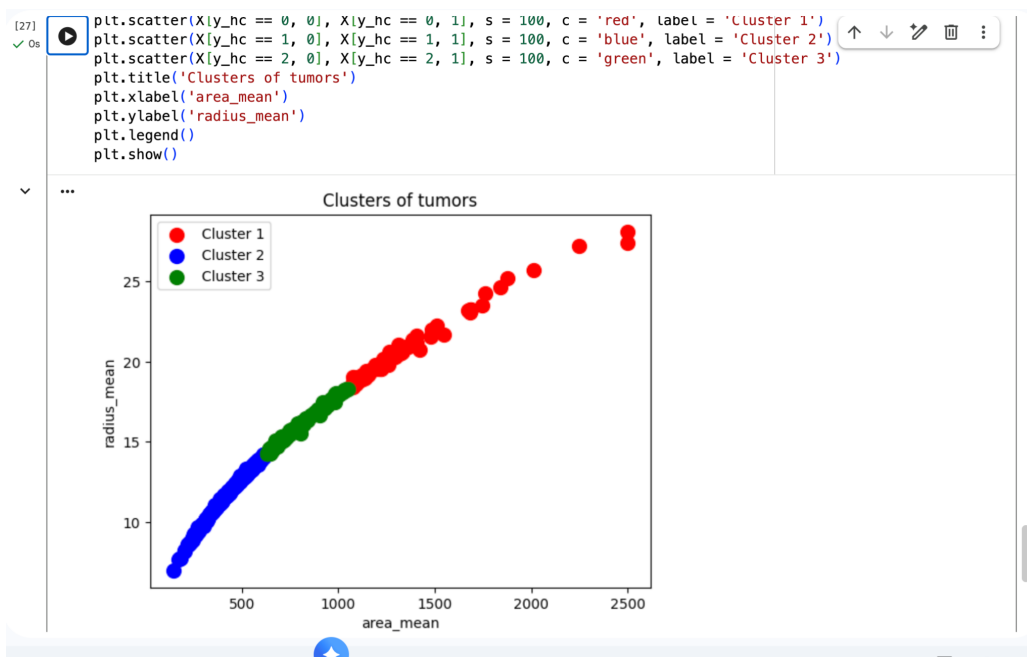
```
... [2 0 0 1 0 1 2 1 1 1 2 2 0 2 1 2 2 2 0 1 1 1 2 0 2 2 2 0 2 0 1 2 0 2 2 2
1 2 1 1 1 0 1 1 0 1 1 1 1 1 1 2 2 1 0 2 1 1 1 1 2 1 1 2 1 1 2 1 1 1 0 1 2 1
1 2 1 2 0 1 1 1 0 0 1 0 2 0 1 2 2 2 1 1 2 0 1 1 1 2 1 1 1 1 1 1 1 1 0 1 1
1 2 1 1 1 1 2 2 2 1 0 0 2 1 1 1 0 2 0 1 2 2 2 0 1 1 1 2 1 1 2 1 1 1 1 1 2
2 1 1 1 1 1 1 2 2 1 1 1 0 0 1 0 2 1 2 2 2 1 1 2 1 1 1 1 2 1 1 0 0 2 1 2
1 2 1 1 2 1 1 1 2 1 1 2 0 2 1 2 0 2 1 2 1 2 1 2 0 1 0 2 2 1 1 1 0 0 1 1
1 2 1 2 1 2 1 1 2 1 1 0 1 2 0 0 2 2 1 1 1 1 0 1 1 1 1 1 0 1 0 2 0 2 0 2 2
2 0 2 2 2 2 0 1 1 1 1 2 1 0 1 2 1 1 0 1 1 0 1 0 2 1 1 1 1 1 1 2 2 1 1 1 1
1 1 2 1 0 1 0 1 1 1 1 1 1 1 2 1 1 1 1 1 2 1 1 1 0 1 0 1 1 2 1 2 2 2 1 1
1 1 2 1 0 1 0 2 1 1 0 1 1 1 2 1 1 1 2 0 2 1 1 1 1 1 1 1 1 1 2 1 0 0 1 0 0
2 2 0 0 1 2 1 1 1 1 1 1 1 1 2 1 1 1 0 1 1 2 0 1 2 1 1 1 1 2 1 1 1 1 1 2
1 2 1 1 1 1 2 2 1 1 2 1 1 1 2 1 1 1 1 1 1 1 2 1 0 0 2 2 1 2 1 2 1 1
2 1 2 2 0 1 0 1 2 1 1 1 1 1 1 2 0 2 1 1 1 1 1 2 1 1 1 2 1 1 1 2 2 1 2 1
2 1 1 2 1 2 0 1 2 1 2 2 1 1 2 1 1 0 0 2 1 1 0 1 1 1 2 2 1 2 1 2 2 1 0 0
1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 0 2 1 1 1 1 2 2 1 1 1 1 1 1 1 1 1 1
1 1 1 2 1 2 1 2 0 0 0 2 0 1]
```

+ Code + Text

The K-Means model grouped the data into 3 clusters, showing clear patterns in the dataset.

e. Visualise the clusters

The clusters are visualized using a scatter plot to show how the data points are grouped based on the selected features.



The clustering graph shows that the data points are distributed along an upward trend, indicating a strong relationship between **area_mean** and **radius_mean**. This is expected because both features represent the size of the tumor, and larger tumors tend to have both higher radius and larger area.

The clusters are grouped along this trend and can be interpreted as different ranges of tumor size:

- **Cluster 1 (blue):** represents tumors with lower values of area and radius, indicating smaller tumors.
- **Cluster 2 (green):** represents tumors with moderate values, indicating medium-sized tumors.
- **Cluster 3 (red):** represents tumors with higher values, indicating larger tumors.

This clustering result shows that the model has successfully grouped the tumors based on size-related characteristics, even though the data points follow a strong linear pattern.

Strategy Formulation(2 Points)

Based on the clustering results, different strategies can be applied for each group.

- Cluster 1: Represents smaller tumors with lower values of area and radius. These cases may require regular monitoring and routine check-ups.
- Cluster 2: Represents medium-sized tumors. These cases may require further medical testing and closer observation.
- Cluster 3: Represents larger tumors with higher values. These cases may indicate more serious conditions and may require immediate medical attention.

These strategies help in understanding tumor severity and support better decision-making in medical analysis.

Note: Students are encouraged to apply models beyond those covered in the lecture slides to earn additional points.

End of Documentation.